

# FYP Interim report

## Project details:

- **Project Title:** Enabling Modularity in IoT: Running WebAssembly Containerized Workloads on Microcontrollers
- **Student name:** Ahmed Ansari
- **Project supervisor:** Professor Stefano Vissicchio

## Project Overview:

This project evaluates the feasibility of deploying containerized control workloads on resource-constrained microcontrollers (MCUs) using WebAssembly (Wasm). The project aims to benchmark containerized solutions against its native counterparts and assess the scalability of this architecture across an IoT network.

## Work completed:

### **1. Implemented a Hardware-in-the-Loop Architecture**

The project utilizes a Hardware-in-the-Loop architecture as a robust testbed for benchmarking WebAssembly solutions against native implementations. This architecture is realized using two ESP32 microcontrollers: one designated as the 'Simulator' and the other as the 'Controller'. The Simulator runs a virtualized physics environment, emulating a thermal system complete with a temperature sensor and a heater actuator.

The Controller is responsible for reading sensor data, processing it via a control algorithm (implemented either natively or within a WASM container), and transmitting a control signal back to the Simulator to modulate the environment. Bidirectional communication between the devices is established via a hardwired analog interface, utilizing the ESP32's internal Analog-to-Digital Converter (ADC) and Digital-to-Analog Converter (DAC) peripherals to exchange continuous signals.

### **2. Analog Communication Layer**

A dedicated analog communication layer manages bidirectional communication between the simulator and the controller. The Simulator transmits simulated temperature data via DAC for the Controller to read via ADC, and the Controller reciprocates with heater commands using its DAC which is read by the Simulator's ADC. Attempts were made to use PWM to relay the Controller command to the Simulator however it was observed that this data transmission was inconsistent and would eventually result in the simulator not receiving commands from the controller.

### 3. Physics Simulation Engine

The Simulator device runs a physics engine designed to model a thermal system with configurable thermodynamic parameters, including thermal mass, ambient temperature, and specific heating or cooling rates. This engine operates on a fixed 50ms real-time update loop to calculate the dynamic energy balance between heater input and environmental loss. The system emulates a virtual thermistor, generating a continuous temperature signal that is transmitted to the Controller node to simulate real-time sensor feedback.

### 4. WebAssembly Runtime (WAMR) Integration

The Controller leverages the WebAssembly Micro Runtime (WAMR) to execute WebAssembly containers directly on the ESP32 hardware. A set of native function bindings serves as a bridge between the isolated WebAssembly sandbox and the device's physical resources, granting the containerized code access to essential capabilities such as logging, timing delays, reading centralized state, and sending commands to the Simulator. Furthermore, the runtime provides functionality to orchestrate the containers, including loading them from storage, instantiation, and resource cleanup. Access to the centralized state is governed by a semaphore, which enforces mutual exclusion for shared resources between containers such as state or control signals.

### 5. Control Algorithms via WASM Containers

Control logic is deployed through modular WASM containers, supporting algorithms ranging from simple Bang-bang logic to complex Proportional-Integral-Derivative (PID) controllers. These tasks are compiled into isolated `.wasm` binaries that can be dynamically loaded by the runtime, enabling the ability to load different control strategies. This facilitates true hot-swappability, allowing for the seamless substitution of algorithms without requiring a system reset.

### 6. Metrics Collection

To rigorously evaluate the comparative performance of native and containerized solutions, a comprehensive suite of metrics was established. This includes profiling WASM execution jitter, control loop variance, and I/O latency. Additionally, end-to-end system lag is measured by the delay between command actuation and the subsequent detection of environmental change via the sensor interface. All collected telemetry is exported in CSV format to facilitate detailed offline analysis and benchmarking.

## **Current work:**

Currently, I am focusing on establishing a rigorous performance baseline by implementing the control algorithms natively in C using the ESP-IDF framework. This native implementation is

essential for me to accurately quantify the specific computational overhead introduced by the WebAssembly containerization.

In parallel, I am designing the architecture for a small-scale IoT network comprising two controller nodes. This setup will allow me to investigate container orchestration strategies, with a specific interest in testing redundancy and distributed hot-swappability. Finally, I am actively conducting a literature survey to ensure my experimental approach remains grounded in the latest research developments.

## **Future work:**

A crucial extension of the project involves exploring how the modularity and hot-swappability of WebAssembly containers can be leveraged within a distributed IoT context . To investigate this, I will develop a small-scale IoT network comprising 2–3 ESP32 Controller nodes communicating with a central Simulator node.

The primary focus of this phase is to implement a robust orchestration platform capable of dynamically deploying containerized control algorithms to available hardware resources. A central objective is the implementation of a state persistence mechanism, ensuring that the last known operational state of a container is preserved and accessible. This architecture will serve as the foundation for testing advanced reliability features such as, state-aware failover, redundancy, and hot-swappability.

I aim to have the fully functioning networked prototype, including the orchestration and failover mechanisms, completed by the end of March. Following the validation of these features, I will proceed to final data analysis and aim to submit the first draft of the final project report by April 10th.